

Round 2 — Take-home (≈6 hours)

Candidate: Akshay Kumar Sinha

Focus: React Native (primary) · minimal backend API (secondary)

Purpose

We want to see how you **ship a small, production-shaped mobile feature**: navigation, state, loading/error/empty states, TypeScript discipline, and a **thin** integration with HTTP APIs. Backend depth is intentionally light—a **small Node server you own** (or equivalent) is enough.

Time budget

Area	Rough guide
App UI + flows	~3.5–4 h
API + wiring	~1–1.5 h
Polish, README, cleanup	~0.5–1 h

Total: about **6 hours**. If you run over, document what you'd do next rather than scope-creeping.

The product slice (scenario)

Build a **Provider “Today’s jobs”** mini-app for a home-services platform:

1. **Jobs list** — Shows today’s assigned jobs (title, time window, area/neighborhood, status: `pending` | `in_progress` | `completed`).

2. **Job detail** — Tap a job: show the same info plus a short “customer note” and a **primary action**:
 - If not completed: “**Mark complete**” (with a required short note, e.g. min 5 characters).
 - If completed: show completion timestamp and the note (read-only).
3. **Mark complete** — Calls your API; on success, update UI (optimistic update is a plus; at minimum refetch or local state update).

Data can be mocked on the server (in-memory array). No real auth, no database, no push notifications.

Technical requirements

Mobile (required)

- **React Native + TypeScript**
- You may use **Expo** or a bare workflow—your choice; state what you used in the README.
- Sensible structure (e.g. screens vs components vs **api** vs types).
- Handle **loading**, **error**, and **empty** states on the list.
- **No** requirement for offline-first or SQLite for this task—optional stretch only (see below).

Backend (required, but keep it tiny)

- **Node.js** (Express or Fastify) or a minimal equivalent you’re comfortable with.
- **Two endpoints minimum**:
 - **GET /jobs** — Returns the list of jobs for “today” (static/mock data is fine).
 - **POST /jobs/:id/complete** — Body: `{ "note": string }`. Validates note length, marks job completed, returns updated job or error.
- In-memory store is fine; restarting the server resets data unless you persist to a JSON file (optional).

Documentation (required)

In your submission repo root, include **README.md** with:

- How to run **API** and **app** (commands, ports).
 - Assumptions and trade-offs (2–4 bullets).
 - What you’d improve with more time (short list).
-

Out of scope (do not build)

- Login / JWT / OAuth
 - Real payments, maps, or live tracking
 - Admin dashboards or web apps
 - CI/CD, EAS builds, or App Store submission
-

Stretch goals (only if time remains)

Pick at most one:

1. **Optimistic UI** for “Mark complete” with rollback on failure.
 2. **React Query (TanStack Query)** or similar for caching/refetch.
 3. **Very small offline queue**: queue the complete action if the request fails; retry when online (can be simulated).
 4. **Light animation** (Reanimated) on completion success—keep it subtle.
-

Deliverables

1. **Git repository** (GitHub/GitLab) with clear commit history (doesn’t need to be perfect, but no single “dump” commit for everything).
2. **README** as above.
3. Short **screen recording** (1–3 min) or screenshots showing list → detail → complete flow.

Send: repo link + Loom/screenshots to the hiring contact by the agreed deadline.

What we evaluate

Dimension	What we look for
RN quality	Clear structure, sensible hooks, typed props, stable lists
UX	Loading/error/empty, readable layout, completion feedback
API integration	Correct HTTP usage, error handling, types aligned with API

Backend	Correct validation, clear routes, easy to run locally
Communication	README: honest scope, trade-offs, “what’s next”

Honesty policy

- You may use **AI tools** if you want—same as day-to-day work.
 - **Disclose** in the README what you used AI for (e.g. “boilerplate Express,” “animation snippet”) and what you wrote/reviewed yourself.
 - We may ask you to **walk through the code** live and extend a small behavior.
-

Optional alignment with our stack (not required)

Our products use React Native (user/provider apps), Next.js web apps, and Node APIs. **You do not need to use or clone this monorepo** for this task. A standalone repo is preferred so we can review your work in isolation.

Questions?

If anything is ambiguous, **make a reasonable assumption**, document it in the README, and proceed. We’re evaluating judgment and execution, not guessing a hidden spec.

Good luck.